Inhaltsverzeichnis

1 Dateien **2**
- 1.0.1 fopen . . . 2
- 1.0.2 fprintf / fscanf . . . 2
- 1.0.3 fputc / fgetc / fputs / fgets . . . 2
- 1.0.4 strtok . . . 2
- 1.0.5 fflush / fclose / remove . . . 2
- 1.0.6 fseek / ftell / rewind . . . 2

2 Präprozessor & Modulkonzept **2**
- 2.0.1 Direktiven . . . 2
- 2.0.2 Include Guard . . . 3
- 2.0.3 Modulkonzept . . . 3
- 2.0.4 Sichtbarkeit . . . 3
- 2.0.5 Prinizipien Modularisierung . . . 3

3 Dynamische Speicherverwaltung **3**
- 3.0.1 Funktionen (`<stdlib.h>`) . . . 3
- 3.0.2 Beispiele . . . 3
- 3.0.3 2D-Arrays . . . 4
- 3.0.4 Typische Fehler . . . 4

4 Verkettete Liste **4**
- 4.0.1 Element einfügen . . . 4
- 4.0.2 Element entfernen . . . 4
- 4.0.3 Liste ausgeben . . . 4
- 4.0.4 Varianten . . . 5
- 4.0.5 Anwendungen & Fehler . . . 5

5 Strings **5**
- 5.0.1 Zuweisung . . . 5

6 Arrays **5**
- 6.0.1 Arrays & Pointer . . . 5
- 6.0.2 Mehrdimensional . . . 5
- 6.0.3 Als Funktionsparameter . . . 6

7 Structs **6**
- 7.0.1 Member-Zugriff . . . 6
- 7.0.2 Funktionsparameter & malloc . . . 6

1 Dateien

Nur Byte-Streams; weitere Organisation Aufgabe des Programms.
Textdatei: Zeichencodes (UTF-8, Latin-1, ASCII); C unterstützt eigentlich nur ASCII.
Binärdatei: Raw bytes, keine Zeichenkodierung; Programm muss Aufbau kennen (Magic Numbers).

1.0.1 fopen

```
FILE *fd = fopen(path, "r");
if (!fd) perror("fopen");
// Modi: r, w, a, rb, wb
```

1.0.2 fprintf / fscanf

```
// int fprintf(FILE *stream, const char *fmt, ...);
fprintf(fp, "Hello %s\n", name);  // ret: Anzahl Zeichen

// int fscanf(FILE *stream, const char *fmt, ...);
fscanf(f, "%d", &x);  // besser: fgets + strtok
```

1.0.3 fputc / fgetc / fputs / fgets

```
int fputc(int c, FILE *stream); // einzelnes Zeichen schreiben
int fgetc(FILE *stream); // einzelnes Zeichen lesen
int fputs(const char *s, FILE *stream); // ganzen String schreiben
char *fgets(char *s, int n, FILE *stream); // Zeichen bis '\n' in String-Buffer lesen, maximal num
    -1 Zeichen

// typisches Lesen mit strtok:
while (fgets(buf, STR_LEN, fd) != NULL) {
  char *tok = strtok(buf, ";\n");
  strcpy(list[i].name, tok);
  tok = strtok(NULL, ";\n");
  list[i].rang = atoi(tok);
}
```

1.0.4 strtok

```
// char *strtok(char *str, const char *delim);
// Erster Aufruf: str; danach NULL. Modifiziert Original!
char *tok = strtok(s, ",");
while (tok != NULL) { tok = strtok(NULL, ","); }
```

1.0.5 fflush / fclose / remove

```
fflush(f);            // Puffer sofort schreiben
fclose(f);            // Puffer leeren + schliessen
remove("data.txt");   // Datei loeschen
```

1.0.6 fseek / ftell / rewind

```
// int fseek(FILE *stream, long offset, int whence);
// whence: SEEK_SET, SEEK_CUR, SEEK_END
fseek(f, 4, SEEK_SET);
long size = ftell(f);
rewind(f);  // == fseek(f, 0, SEEK_SET)
```

2 Präprozessor & Modulkonzept

Build-Pipeline: Präprozessor → Compiler (.o) → Linker → Loader

2.0.1 Direktiven

```
#include <stdio.h>        // Systemverzeichnis
#include "datei.h"        // Projektverzeichnis

#define ANZAHL 100
#define QUADRAT(X) ((X)*(X))  // Klammern wichtig!
// QUADRAT(3+5) ohne Klammern -> 3+5*3+5=23 statt 64
```

```
#ifdef DEBUG
printf("debug\n");
#endif  // Aktivieren: gcc -DDEBUG ...
```

2.0.2 Include Guard

```
#ifndef MEINE_DEFS_H
#define MEINE_DEFS_H
// Inhalt der Header-Datei
#endif
```

2.0.3 Modulkonzept

```
// doit.h
#ifndef DOIT_H
#define DOIT_H
#define MAX 1000
int doit(int b);
#endif

// doit.c
#include "doit.h"
int doit(int b) { return MAX + b; }

// gcc -c main.c && gcc -c doit.c && gcc -o prog *.o
```

2.0.4 Sichtbarkeit

```
int globalFunc(int a) { ... }        // ueberall sichtbar
static int lokalFunc(int b) { ... }  // nur im Modul
```

2.0.5 Prinizipien Modularisierung

Geringe Vernetzung	wenige Schnittstellen
Schwache Koppplung	schmale Schnittstellen
Hohe Kohäsion	inhaltliche Einheit - wohledfinierte Aufgabe
Explizite Schnittstelle	Kommunikation mit Parameterübergrabe
Information Hiding	nur global was muss; Verstecken Implementationsdetails; Trennung Interface & Implementation

3 Dynamische Speicherverwaltung

Code	Programmcode
Daten	globale/statische Variablen
Heap	dynamisch alloziert
Stack	lokale Variablen (temporär)

Keine Adresse lokaler Variablen zurückgeben – Stack wird nach `return` abgebaut!

3.0.1 Funktionen (`<stdlib.h>`)

```
void *malloc(size_t size);              // anfordern
void  free(void *ptr);                  // freigeben
void *calloc(size_t num, size_t size); // anfordern + 0
void *realloc(void *ptr, size_t size); // Groesse anpassen
```

Immer `NULL`-Check nach `malloc`; `ptr = NULL` nach `free`.

3.0.2 Beispiele

```
// Array
int  *arr = malloc(n * sizeof(int));
free(arr);
arr=NULL;
// Struct
Zeit *pZ  = malloc(sizeof(Zeit));
pZ->h = 4;
free(pZ);

// String: erst grosser Buffer, dann passend
char *buf = malloc(256);
```

```c
fgets(buf, 256, stdin);
char *s = malloc(strlen(buf) + 1);
strcpy(s, buf);  free(buf);  free(s);
```

3.0.3 2D-Arrays

```c
// Variante 1: 1D, Index manuell
int *a = malloc(rows * cols * sizeof(int));
a[i*cols + j] = ...;

// Variante 2: Array von Zeigern (a[i][j])
int **a = malloc(rows * sizeof(int *));
for (int i = 0; i < rows; i++)
a[i] = malloc(cols * sizeof(int));
for (int i = 0; i < rows; i++) free(a[i]);
free(a);
```

3.0.4 Typische Fehler

```c
double *p = malloc(n * sizeof(n));       // FALSCH: sizeof(n)
double *p = malloc(n * sizeof(double)); // korrekt
Datum  *d = malloc(10 * sizeof(Datum*));// FALSCH: sizeof(Datum*)
Datum  *d = malloc(10 * sizeof(Datum)); // korrekt

int *ptr = malloc(...);
ptr = &array[0];  // Memory Leak: Heap verloren!
```

4 Verkettete Liste

Arrays: Lücken beim Entfernen ⇒ teures Umkopieren. Listen: individuelle Allokation pro Element.

```c
typedef struct ListElem_s {
  char name[16]; char first[16]; int rank;
  struct ListElem_s *next;
} ListElem;
```

4.0.1 Element einfügen

```c
// Mittendrin (nach item): Reihenfolge wichtig!
newItem->next = item->next;
item->next    = newItem;

// Am Anfang:
newItem->next = head;  head = newItem;

// Am Ende:
while (item->next != NULL) item = item->next;
item->next = newItem;  newItem->next = NULL;
```

4.0.2 Element entfernen

```c
// Nach item:
delPtr = item->next;  item->next = delPtr->next;
free(delPtr);

// Am Anfang:
delPtr = head;  head = head->next;  free(delPtr);

// Am Ende:
while (delPtr->next != NULL)
{ delPtr = delPtr->next; item = item->next; }
item->next = NULL;  free(delPtr);
```

4.0.3 Liste ausgeben

```c
for (ListElem *it = head; it != NULL; it = it->next)
printf("%s %s\n", it->first, it->name);
```

4.0.4 Varianten

Ankerelement: Dummy-Head eliminiert Sonderfall am Anfang.
Tail-Zeiger: append O(1) statt O(n).
Doppelt verkettet: `prev`-Zeiger für Navigation in beide Richtungen.

4.0.5 Anwendungen & Fehler

Queue (FIFO): enqueue / dequeue an verschiedenen Enden.
Stack (LIFO): push / pop am selben Ende.
Binärbaum: left/right-Zeiger, Suche O(log n).

- Speicher zu früh freigeben ⇒ Pointer verloren
- Falsche Reihenfolge beim Umhängen ⇒ Liste abgetrennt
- `free()` vergessen ⇒ Memory Leak

5 Strings

```
char  c      = 'A';      // Char-Literal: 1 Zeichen, Typ char
char *s      = "Hallo"; // String-Literal: Zeiger, read-only
char  arr[64];           // Array: fixer Speicher, beschreibbar
```

5.0.1 Zuweisung

```
char arr[] = "Max";       // OK -- bei Deklaration
arr[0] = '\0';            // OK -- einzelnes Zeichen
strcpy(arr, "Max");       // OK -- ganzen String kopieren
arr    = "Max";           // FEHLER -- Array ist kein l-value
arr[0] = "\0";            // FEHLER -- String-Literal != char
```

	`'\0'`	`"\0"`
Typ	`char` (1 B)	`char*`
Wert	Zeichen-Wert	Adresse
Grösse	1 Byte	2 Bytes

Situation	Typ
Struct-Member	`char name[N]`
Dynamische Länge	`char*` + malloc
Funktionsparameter (lesen)	`const char*`

`char name[N]` wann immer möglich – kein malloc, kein free, keine Leaks.

```
typedef struct { char first[64]; char name[64]; } Participant;
memset(p, 0, sizeof(Participant));  // alles auf 0/NULL
```

6 Arrays

```
int a[5];                          // uninitialisiert
int a[5] = {1, 2, 3, 4, 5};       // vollstaendig
int a[]  = {1, 2, 3, 4, 5};       // Groesse auto
int a[5] = {1, 2};                 // Rest = 0
int a[5] = {0};                    // alles 0
int *a   = (int[]){1,2,3};         // Compound Literal (C99)

a[2] = 42;   int x = a[2];   *(a+2) = 42;  // aequivalent
```

6.0.1 Arrays & Pointer

`a == &a[0]`; `a[i]` ≡ `*(a+i)`

```
int *p = a;  p[2];  *(p+2);  // alle gleich wie a[2]
```

6.0.2 Mehrdimensional

```
int m[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
m[1][2] = 42;
```

6.0.3 Als Funktionsparameter

```
void foo(int *a, int n);     // als Pointer
void foo(int a[], int n);    // aequivalent
void foo(int a[10], int n); // Groesse wird ignoriert
```

Arrays werden **immer als Pointer** übergeben – Länge separat mitgeben.

7 Structs

```
typedef struct { char name[16]; int rank; } Person;

Person p;                                   // uninitialisiert
Person p = {"Muster", 1};                  // positionsbasiert
Person p = {.name="Muster", .rank=1};      // designated init
```

7.0.1 Member-Zugriff

```
p.rank = 1;          // direkt: . Operator
p.name[0];           // Array-Member

Person *ptr = &p;
ptr->rank = 1;       // ueber Pointer: -> Operator
(*ptr).rank = 1;     // aequivalent, unueblich
```

`p.member` wenn `p` ein Struct ist.
`p->member` wenn `p` ein Pointer ist ≡ `(*p).member`

7.0.2 Funktionsparameter & malloc

```
void foo(Person p);    // Kopie -- lokale Aenderungen
void foo(Person *p);   // Pointer -- wirkt auf Original

Person *p = malloc(sizeof(Person));
p->rank = 1;  free(p);
```